12.4.1. Creating an artifact (something that's easy to deploy)

First, you need to create something that will be sent to the server and become the program that runs there. That something is called an *artifact*.

Given that you have a compiled version of the application already, you can create an artifact from the compiled program. From Cake, the Node.js `execFile` command allows you to call out to an external program that can compress the compiled program and put it in a single file ready to be deployed:

```
createArtifact = (path, version, callback) ->
  execFile "tar", ["-cvf", "artifact.#{version}.tar", path], (e, d) ->
    callback()

task 'artifact', 'build the artifact', ->
  version = fs.readFileSync './VERSION', 'utf-8'
  createArtifact 'compiled', version, ->
    console.log "done. artifact.#{version}.tar generated"
```

Execute the command-line program tar to create the artifact. If your operating system doesn't have a tar command, then consult the relevant documentation and use an equivalent program.

The build version is kept in a file. As before, in a build it's generally acceptable to use synchronous file loads if the build can't run at all without those files.

When you run `cake artifact`, a file containing all of the compiled files is created. Create a file called VERSION in the same directory as the Cakefile and put the number `1` in it. Now when you invoke `cake` with the `artifact` task, it will use this version number when generating the artifact:

```
> cake artifact
# done. artifact.1.tar generated
```

How about the manifest? Why do you need one and how do you create it?

12.4.2. Creating a manifest (something that tells your artifact where it is)

Consider that in development you run the application on localhost on port 8080. Looking at listing 12.2, you notice that this value is hardcoded: